

A Case for Simulation-Driven Resilience in Agent-First Data Systems

Aleksey Charapko
University of New Hampshire
Aleksey.Charapko@unh.edu

Murat Demirbas Akshat Vig
MongoDB
{Murat.Demirbas;Akshat.Vig}@mongodb.com

Abstract—AI agents are replacing humans as the primary clients of modern data systems, and are bringing with them a qualitative shift in workload characteristics. Agents retry aggressively with mutated queries, spawn bursty parallel sub-tasks, and hold transactions open while awaiting external LLM responses. These behaviors systematically violate assumptions baked into every layer of modern data systems (from execution control and admission policies to caching layers and concurrency control) and create fertile ground for *metastable failures*: self-sustaining states of degraded throughput that persist even after the triggering overload subsides. We argue that the community needs a simulation-driven methodology to systematically discover and prevent agent-induced failures across the data system stack *before* facing production incidents. We demonstrate this approach on Execution Control Systems (ECS), where we show how discrete-event simulation with MESSI reveals failure modes invisible to benchmarks, and outline a research agenda extending the methodology to other data system services and components.

I. INTRODUCTION

AI agents are on the verge of replacing humans as the primary clients of data systems. They autonomously compose multi-step workflows that query databases, analyze results, and iteratively refine their approach [1], [2]. Solutions like the Model Context Protocol (MCP) [3] are standardizing how agents connect to data sources and accelerating the transition from human-driven to agent-driven database traffic. Unfortunately, data systems are not ready for this phase shift transition.

Every layer of a modern data system embeds assumptions about how clients behave: execution control assumes stationary arrivals, caching assumes temporal locality, and concurrency control assumes bounded hold times. Agents violate all of these simultaneously. Adapting data systems for agentic workloads is a broad challenge: recent work has proposed redesigning query interfaces, transaction semantics, and storage layers to natively support agentic speculation [4]. These are important directions, but none of them matter if the system cannot stay up.

Thus, we focus our deepest analysis on the **Execution Control System (ECS)**, the component mediating resource contention at the backend through queues, concurrency limits, and scheduling decisions. Concretely, the ECS sits between accepted client requests and the execution engine: it manages priority enforcement and compute resource allocation among executing requests. Typically, ECS uses a bounded pool of execution *tickets* to manage concurrency and resource distribution. ECS places admitted requests into one or more queues

with different ticket allocation based on perceived priority and dispatches them to worker threads or storage I/O slots. The queues and per-queue ticket budgets allow prioritizing some requests over others when resources are constrained. To cope with heterogeneous workloads, production ECS designs layer on priority- or runtime-aware multi-queue scheduling [5], timeout-based load shedding for stalled work, and backpressure signals that throttle upstream admission when downstream resources saturate [6]. The ECS is the critical first domino: when it fails, every other subsystem degrades. But the threat does not stop at ECS – the same agent behaviors that destabilize execution control also poison caching layers, buffer pools, and lock managers, and the simulation methodology we develop here generalizes to each of them.

Just like many other components of data systems, today’s ECS designs are built on assumptions that agents systematically violate: stationary arrival processes, bounded service-time distributions, graceful client backoff, and independent requests. Although agents interact with data systems through driver libraries, connection pools, and middleware that impose some client-side discipline, they still exhibit characteristics that are qualitatively different from traditional clients:

- **Aggressive retries with mutation.** When a query fails, an agent’s reasoning loop treats this as a problem to solve, retrying the query, rewriting it, or spawning alternatives. Even with standard retry middleware, the agent’s tendency to *alter* the query on each attempt produces a stream of novel requests that evade per-query deduplication.
- **Non-deterministic query generation.** Agents generate SQL or API calls via LLM inference. Two invocations of the same logical task can produce radically different queries with different resource profiles, making *a priori* task classification nearly impossible.
- **Bursty parallelism.** Agentic frameworks routinely “fan out” work to sub-agents. A single user request can spawn dozens of concurrent database operations in milliseconds, then go idle.
- **External blocking.** Agents may pause mid-transaction to invoke external LLM APIs, dispatch sub-agents, or call other tools, holding database resources (tickets, connections, locks) across reasoning steps. Effective hold time becomes query-execution time *plus* inference latency multiplied by the number of reasoning steps spent inside the

transaction, breaking pool-sizing and ticket-budget assumptions calibrated for query execution alone.

These characteristics (documented in early empirical studies of agentic database access [4]) combine to produce sudden bursts of unpredictable, resource-intensive load that we call *agentic spikes*. Liu et al. [4] report that at Neon, agents created $20\times$ more branches and performed $50\times$ more rollbacks compared to humans, confirming that the volume and volatility of agent-generated operations far exceeds what human-paced scaling heuristics anticipate. Agentic spikes are also qualitatively different from traditional load spikes. They are discontinuous, self-reinforcing, and correlated through the agent’s reasoning loop. This combination is what pushes ECS overloads from transient spikes that drain after the trigger subsides into *metastable collapse*: a self-sustaining degraded regime in which the very mechanisms designed to protect the system (retries, load shedding, queue management) close feedback loops with the agent and keep the system stuck [7].

In sum, data systems were designed for a world where clients behave predictably, and agents do not. We argue that agentic workloads constitute a *phase change* in database workload characteristics, a qualitative shift that invalidates assumptions embedded across the data system stack. Patchwork fixes will not suffice. We propose *simulation-driven resilience*: systematic exploration of the agent-database boundary using discrete-event simulation to discover and prevent these failures before production incidents force reactive fixes. We demonstrate this approach on ECS (speculation-aware scheduling and admission control co-design) and outline how the same simulation framework can be applied to caching, buffer management, and more. Finally, we propose empirically-grounded workload generators built on a *stateful frustration model* that captures how agents escalate from well-behaved clients to adversarial cache-busters.

II. AGENT-INDUCED METASTABILITY

Metastability is a failure mode in which a distributed system becomes trapped in a self-sustaining state of degraded performance: unlike transient overload, metastable failures persist even after the initial trigger is removed, because resilience mechanisms (retries, queues, timeouts) turn inward and sustain the degraded state [7]. We argue that AI agents amplify every known metastability trigger and introduce entirely new feedback loops. Figure 1 illustrates the core loops that arise when agentic clients interact with an ECS.

A. The Agentic Retry Storm

Traditional clients implement well-defined retry policies (exponential backoff, jitter, circuit breakers), and human users provide natural backpressure: they wait, switch tasks, or give up. Agents behave differently even when equipped with standard retry middleware. Agents not only retry *faster* (rate limiters can constrain that) but also retry *differently each time*. An LLM agent encountering a timeout may rewrite the query, break it into smaller queries, or escalate to a more expensive operation. Each retry appears as a novel request with an

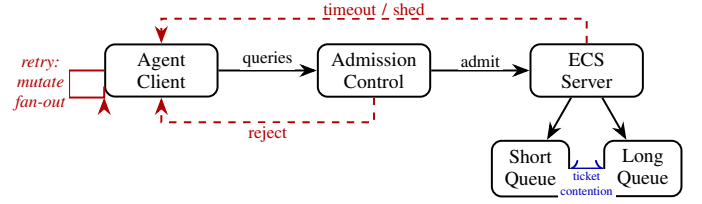


Fig. 1. Agent-ECS feedback loops. Solid arrows show the request path. Dashed red arrows show the feedback that drives metastability: rejected or timed-out requests return to the agent, which retries with mutated queries or parallel fan-out, amplifying load. Ticket contention between queues (blue) creates an internal feedback loop within the ECS.

unpredictable resource profile invisible to deduplication or per-query throttling.

B. The Amplified Convoy Effect

ECS designs use multi-queue architectures that classify tasks by *elapsed wall-clock time*: a task exceeding a threshold (e.g., 10 ms) is demoted from the high-priority short-task queue to the lower-priority long-task queue. Agentic workloads may poison this classification.

Agents that call external LLM APIs mid-transaction hold database tickets for seconds while awaiting inference results, even though the actual database work takes only microseconds. The ECS may classify these as long-running tasks despite their relatively low CPU and IO overhead. Unfortunately, these misclassified tasks may hold locks on data needed by other operations, increasing overall data conflicts and potentially forcing other short tasks to wait longer and be misclassified as long tasks, creating a feedback loop. ECS may also load-shed long tasks to save resources; creating too many artificial long tasks increases the error rate observed by the agents, contributing to the agentic retry storms.

C. Novel Feedback Loops

Beyond amplifying known failure modes, agentic workloads create new feedback loops.

Retry–queue–ticket exhaustion spiral. Agent retries flood the admission queue. Queue growth exhausts the short-task ticket pool. New requests are either dropped or demoted to the long-task queue. Agents interpret drops as errors and retry *more aggressively*, closing the loop.

Query mutation–classification confusion. An agent’s retry loop rewrites a failed short query as a complex join or aggregation. The ECS correctly classifies this as a long task and demotes it. The agent, receiving a timeout on the demoted query, rewrites it *again*. The ECS is now tracking a stream of “independent” requests that are actually a single agent’s frustrated retry loop, each with a different resource profile.

Multi-agent lock contention cascade. When multiple agents collaborate on shared state, they generate non-deterministic lock acquisition orders. This reduces throughput and intensifies contention through agents spawning additional parallel attempts.

The feedback loops described above represent a fundamental mismatch between the world model that current ECS

designs are built for (stationary arrivals, bounded service times, graceful backoff, independent requests [8], [9]) and the threat model that agentic workloads present.

III. SIMULATING THE AGENT-DATABASE BOUNDARY

The complexity of agent-database interactions (with their feedback loops, non-stationary arrivals, and correlated retry behavior) exceeds what analytical queuing models can capture. Traditional approaches like Markov chains or queueing theory assume stationary processes and independent arrivals [10]. We need tools that can model the *dynamic* interplay between agent behavior and system state.

A. MESSI: Discrete-Event Simulation for Metastability

In our preliminary work, we developed MESSI (MEtaStability SIMulator) [11], a discrete-event simulation framework designed to explore metastability dynamics in distributed systems. MESSI models systems as directed graphs where **Logic Nodes** implement routing and state policies, and **Processors** simulate temporal resource constraints (queues, I/O delays, network latency). Individual work items (**QItems**) carry state as they traverse the graph. Leveraging this graph abstraction enables MESSI to model and investigate the metastability dynamics of any service, or composition of services, in a modern distributed data system stack.

B. What Simulation Reveals That Benchmarks Cannot

Static benchmarks like TPC measure steady-state throughput under fixed workload mixes, but metastable failures are *transient* phenomena that emerge from specific event sequences and may never appear in steady-state measurement. In our preliminary ECS work [11], MESSI’s tick-by-tick simulation revealed that two independently reasonable policies, per-queue ticket probing for short and long queues, create a metastable feedback loop when composed: the long queue’s ticket growth steals CPU from short tasks, forcing the short queue to escalate in turn, until both hit their limits. This failure mode is invisible to steady-state analysis.

Even when an agentic benchmark does provoke a collapse on a real system, the experiment yields throughput and latency curves but does not provide a *causal explanation*. The queue depths, ticket histories, and retry trees that drive metastability are not exposed, and the instrumentation needed to see them perturbs the very dynamics under study. Every candidate fix (a different ticket policy, shedding heuristic, or admission strategy) further demands rebuilding and redeploying the production stack, making it impractical to sweep design alternatives. MESSI inverts this calculus. It exposes the full internal state of system components at every simulation tick, and enables root-cause analysis. Experiments are deterministic and replayable, so a metastable trigger discovered once can be re-run against alternate ECS designs to see which survive. The simulation environment enables users to test hypotheses, compare alternatives, and converge on a robust design before any production change is committed.

Agentic workloads bring further challenges for observability as the space of possible triggering sequences is vastly larger: an agent’s retry strategy, query mutation pattern, and parallelism decisions interact with queue bounds, ticket policies, cache eviction rules, and lock scheduling heuristics. Systematically exploring this space requires the controlled repeatable experimentation that only simulation provides.

C. Modeling Agentic Clients

Simulation is only as good as its workload model, but here we face an empirical gap: no equivalent of TPC exists for agentic workloads. Early characterization [4] reveals key properties (retry intensity, query mutation, fan-out, external blocking) but these have not been codified into reproducible generators. To close this gap, we propose extending MESSI with a **Simulated Agent Client**: a Logic Node that emits synthetic agentic traffic exhibiting the four characteristics identified in Section II.

The client is built around a *stateful frustration model*. It maintains a frustration index that increments on each timeout, rejection, or queue demotion and resets on success. As frustration rises, the agent escalates: at low levels it retries with backoff; at moderate levels it *rewrites* the query, drawing from a heavier service-time distribution and shifting from point lookups toward broader scans that randomize its working set and break spatial locality; at high levels it *fans out* into parallel QItems across non-overlapping data partitions. Each emitted QItem carries a trace ID linking it to the originating task, so MESSI can reconstruct the full retry tree and expose the cross-subsystem feedback loops that drive metastable collapse. Frustration thresholds, fan-out degree, service-time distributions, and external-blocking probability are all configuration parameters, so the same model spans the spectrum from well-behaved clients to adversarial cache-busters.

To ground the synthetic models in real agent behavior, we propose controlled studies in which modern LLM agents are given open-ended analytical tasks against complex reference datasets, with execution paths continuously traced. Measuring the semantic distance of query mutations, the frequency of fan-outs, and the duration of external LLM pauses will yield the parameters needed to calibrate the generator.

Finally, to validate our simulation results with that of real systems, we propose a *closed-loop validation* methodology: use MESSI to sweep the parameter space (frustration levels, fan-out degrees, ticket limits), identify configurations that trigger metastable collapse, and then replay those exact workloads against instrumented deployments of production databases such as MongoDB. By correlating simulation predictions with production telemetry (buffer pool eviction rates, lock wait queue depths, disk I/O spikes, throughput degradation), we aim to validate the model’s fidelity and give engineers the concrete reproducible evidence of agent-induced metastability needed to justify architectural changes.

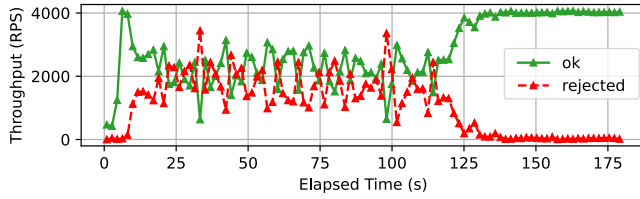


Fig. 2. Admission Control and ECS can have destructive interference.

IV. WHAT CAN SIMULATION ATTACK?

We now outline a research agenda that applies simulation-driven resilience across the data system stack: first to ECS internals (where we have preliminary results), then to deeper subsystems.

A. ECS for Agentic Speculation

Liu et al. characterize agentic database interaction as *agentic speculation* [4]: agents issue high-throughput, exploratory queries (many of them redundant) to ground themselves before formulating solutions. Only 10–20% of sub-plans are unique across parallel attempts, and queries progress through distinct phases from coarse metadata exploration to targeted solution formulation. This two-phase pattern creates a distinctive ECS challenge: exploratory queries consume all short-task tickets, and when the agent transitions to solution formulation, those heavier queries get classified as long tasks and starved. MESSI can model this by implementing agent Logic Nodes that transition between exploration and formulation modes, enabling systematic study of whether ECS policies should differentiate between phases and what signals could detect the transition.

B. Admission Control and ECS Co-Design

In our preliminary work [11], we found a surprising result: combining admission control with execution control can cause *destructive interference*, where admission control drops short and long tasks indiscriminately, reducing short-task goodput below what the ECS alone would achieve. We illustrate this in Figure 2, where the workload spikes at 5 seconds of elapsed time, causing immediate activation of Admission Control even though the ECS would have managed the load just fine after a short adjustment. However, with admission control dropping work, ECS cannot make the necessary adjustments to resource allocations and priorities, leaving the system stuck in reduced-goodput mode until the workload subsides. Under agentic workloads, this interaction becomes far more complex.

Admission control operates at the network edge and cannot distinguish an agent’s first attempt from its fifth retry, nor whether a rejection will cause the agent to back off or escalate. MESSI’s composable graph architecture can model both admission control and ECS as interacting components in the same simulation graph, enabling researchers to explore agent-aware admission policies and evaluate whether such policies improve or worsen metastability risk when composed with the ECS’s internal queue management.

C. Beyond Execution Control: Caching and Concurrency

Buffer pool and caching layers. Frustrated agents naturally behave as cache adversaries. When an agent’s initial query fails, its mutated retries abandon hot well-optimized data paths in favor of cold data subsets. This working-set randomization causes buffer pool thrashing, rapidly evicting useful pages and driving up disk I/O. By modeling the buffer pool as a MESSI Processor with LRU or clock-based eviction, we can test agent-aware cache isolation policies. For instance, we can pin stable application pages while allowing exploratory agentic queries to compete only within a bounded eviction budget.

Lock management and concurrency control. Agents generate queries non-deterministically, producing unpredictable lock acquisition orders across concurrent sub-tasks. Combined with long external blocking pauses (agents waiting on LLM inference while holding locks), this creates fertile ground for cascading lock contention and transaction starvation. Modeling the lock manager within MESSI enables us to observe how these non-stationary patterns interact with deadlock detection heuristics and explore whether agent-aware lock scheduling can prevent contention cascades.

V. RELATED WORK

Metastability. Huang et al. [7] established metastability as a first-class failure mode in distributed systems; Isaacs et al. [12] extended this with modeling and simulation approaches. Habib et al. [10] apply queuing-based analysis to predict metastable failures in replicated storage. Our work builds on these foundations but focuses on a new trigger: agentic workloads.

ECS and overload control. SEDA [13] introduced staged, event-driven admission control; Shenango [14] and Shinjuku [5] achieve low tail latency through kernel-bypass scheduling. Breakwater [6] applies credit-based admission for RPCs. These systems assume well-behaved clients with predictable backoff, which agents violate.

Agent-first data systems. Liu et al. [4] propose a comprehensive agent-first architecture covering query interfaces, probe optimization, and storage. Our work is complementary: we focus on the resilience and resource-management layers that underlie their proposed architecture, and advocate simulation-driven methodology for systematically discovering agent-induced failures across the stack.

VI. CONCLUSION

Agents are not faster humans, they are a new threat model for data systems. Their behavior violates assumptions across the data system stack and turns protective mechanisms into amplifiers of failure. We have argued that simulation-driven resilience is how the community should respond: use discrete-event simulation to find these failures before production does. We demonstrated the approach on ECS, argued that it extends to caching and concurrency control, and outlined the workload models and validation methodology needed to make progress.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023.
- [2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [3] Anthropic, “Model context protocol,” <https://modelcontextprotocol.io>, 2024, open standard for connecting AI agents to data sources and tools.
- [4] S. Liu, S. Ponnappalli, S. Shankar, S. Zeighami, A. Zhu, S. Agarwal, R. Chen, S. Suwito, S. Yuan, I. Stoica, M. Zaharia, A. Cheung, N. Crooks, J. E. Gonzalez, and A. G. Parameswaran, “Supporting our AI overlords: Redesigning data systems to be agent-first,” *arXiv preprint arXiv:2509.00997v2*, 2025.
- [5] K. Kaffes, T. Chong, J. T. Humphries, A. Belay, D. Mazières, and C. Kozyrakis, “Shinjuku: Preemptive scheduling for μ second-scale tail latency,” in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 345–360.
- [6] I. Cho, A. Saeed, J. Fried, S. Jin, and A. Belay, “Overload control for μ -scale RPCs with breakwater,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 299–314.
- [7] L. Huang, M. Magnusson, A. Bangalore Muralikrishna, S. Estyak, R. Isaacs, A. Aghayev, T. Zhu, and A. Charapko, “Metastable failures in the wild,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 73–90.
- [8] D. Yanacek, “Using load shedding to avoid overload,” AWS Builders Library, 2019.
- [9] M. Welsh and D. Culler, “Adaptive overload control for busy internet servers,” in *4th USENIX Symposium on Internet Technologies and Systems (USITS 03)*, 2003.
- [10] F. Habib, T. Llorido-Botran, A. Showail, D. C. Sturman, and F. Nawab, “Msf-model: Queuing-based analysis and prediction of metastable failures in replicated storage systems,” in *2024 43rd International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2024, pp. 12–22.
- [11] A. Charapko, M. Demirbas, M. Broadstone, D. Gomez Ferro, and A. Vig, “Towards designing an execution control system with metastability resilience,” 2025, under review.
- [12] R. Isaacs, P. Alvaro, R. Majumdar, K. Kumar, M. Reddy, M. Salamati, and S. Soudjani, “Analyzing metastable failures,” in *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, 2025, pp. 172–178.
- [13] M. Welsh, D. Culler, and E. Brewer, “SEDA: An architecture for well-conditioned, scalable internet services,” in *ACM SIGOPS Operating Systems Review*, vol. 35, no. 5, 2001, pp. 230–243.
- [14] A. Ousterhout, J. Fried, J. Behrens, A. Belay, and H. Balakrishnan, “Shenango: Achieving high CPU efficiency for latency-sensitive data-center workloads,” in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 361–378.